

PseudoCode:

```
int swap_vector(array& row) {  
    swap_count = 0  
    n = row size  
    positions = empty // Initialize a map to track positions of elements  
  
    for (i from 0 to n - 1) {  
        positions[row[i]] = i // Populate the map with element indices  
    }  
  
    for (base from 0 to n - 2) {  
        if (the value at base is already next its pair)  
            Skip to the next iteration  
  
        pair_value = PAIRS[row[base]] // Find the pair value for the current element  
  
        if(pair_value is found in positions  
            point = positions[pair_value]  
  
        if(point > base + 1) {  
            swap(row[base + 1], row[point]) // Perform the swap  
            swap_count = swap_count + 1 // Increase count  
            // Update the positions map after the swap  
            positions[row[base + 1]] = base + 1  
            positions[row[point]] = point  
        }  
    }  
    return swap_count // Return the total count of swaps  
}
```

Calculating Time Complexity:

Costs of Initialization = 3

- swap_count = 1
- n = 1
- positions = 1

Cost of populating positions = n

- for loop = n
- nested function: 1 (initializing row at i)

Cost of swapping elements = $\frac{n}{2}$ (3)

- for loop = $\frac{n}{2}$
- nested functions = 1(initializing pair_value) + 1 (finding pair_value) + 1 (performing swap)

➤ **$T(n) = 3 + n + \frac{n}{2}$ (3)**

Best Guess

- $T(n)$ seems to resemble the linear efficiency class of $O(n)$

Finding a 'c' such that $T(n) \leq c * f(n)$

- **$3 + n + \frac{n}{2}$ (3) $\leq c * (n)$**
- **$((3 + n + \frac{n}{2}) (3)) / n \leq c$**
- **$\frac{3}{n} + 1 + \frac{3}{2} \leq c$**
- Since n must be ≥ 2 , $c = 4$

Base case:

- n = 2, the size of the row cannot be less than 2.
- $T(2) = 3 + (2) + \frac{2}{2}$ (3)
- $T(2) = 8$
- $c * f(n) = (4)(2) = 8$
- $8 \leq 8$, therefore our base case holds.

Inductive Step:

- $T(n + 1) \leq c * (n + 1)$
- **$3 + n + 1 + \frac{n+1}{2}$ (3) $\leq 4 * (n + 1)$**
- **$2.5n + 5.5 \leq 4n + 4$**
- The inductive step holds.
- Thus, $T(n)$ belongs to $O(n)$

Calculating Space Complexity:

Costs of Initialization = 3

- swap_count = 1
- n = 1
- positions = 1

Cost of populating positions = n

- populating positions = n

Cost during swapping = $\frac{n}{2} (2) = n$

- nested functions in for loop of $\frac{n}{2}$: 1(initializing pair_value) + 1 (initializing point)

➤ **$T(n) = 3 + n + n$**

➤ **$T(n) = 2n + 3$**
